

Relatório de Modularização de Código e Análise Comparativa de LLMs

1. Introdução

Este relatório apresenta uma análise comparativa da modularização de código Python gerado por duas Large Language Models (LLMs), ChatGPT (simulado com GPT-5) e Gemini (simulado com Gemini 3 Flash Preview), para um problema de monitoramento de potência de sinal recebido em dBm, específico para o contexto de Telecomunicações. O objetivo é avaliar a capacidade dessas IAs em produzir código bem estruturado e modularizado, seguindo princípios de engenharia de software, e comparar com um pseudo-código autoral.

2. Parte 1: Pseudo-código Autoral

A seguir, é apresentado o pseudo-código desenvolvido para resolver o problema de monitoramento de sinal, detalhando as etapas funcionais e a decomposição em funções com responsabilidades únicas.

Plain Text

Função Principal: ProcessarMedicoesSinal(lista_medicoes_brutas)

Inicializar:

```
lista_medicoes_validas = []
valores_descartados_contagem = 0
valores_criticos_contagem = 0
sequencia_critica_maxima = 0
sequencia_critica_atual = 0
```

Para cada medição em lista_medicoes_brutas:

Se ValidarMedicao(medição) for verdadeiro:

Adicionar medição à lista_medicoes_validas

Senão:

Incrementar valores_descartados_contagem

Se lista_medicoes_validas estiver vazia:

Retornar "Nenhuma medição válida para processar."

Calcular Estatísticas:

```
media = CalcularMedia(lista_medicoes_validas)
minimo = EncontrarMinimo(lista_medicoes_validas)
maximo = EncontrarMaximo(lista_medicoes_validas)
desvio_padrao = CalcularDesvioPadrao(lista_medicoes_validas)
```

Para cada valor em lista_medicoes_validas:

status = ClassificarMedicao(valor)

Se status for "Crítico":

Incrementar valores_criticos_contagem

Incrementar sequencia_critica_atual

Senão:

Redefinir sequencia_critica_atual para 0

Se sequencia_critica_atual > sequencia_critica_maxima:

sequencia_critica_maxima = sequencia_critica_atual

Gerar Relatório:

Imprimir "Relatório de Monitoramento de Sinal"

Imprimir "Total de medições brutas: " + tamanho(lista_medicoes_brutas)

Imprimir "Medições válidas: " + tamanho(lista_medicoes_validas)

Imprimir "Medições descartadas: " + valores_descartados_contagem

Imprimir "Média: " + media + " dBm"

Imprimir "Mínimo: " + minimo + " dBm"

Imprimir "Máximo: " + maximo + " dBm"

Imprimir "Desvio Padrão: " + desvio_padrao + " dBm"

Imprimir "Medições Críticas: " + valores_criticos_contagem

Imprimir "Maior sequência consecutiva de Críticos: " + sequencia_critica_ma

Função ValidarMedicao(valor):

Se valor é numérico E valor ≥ -120 E valor ≤ 0 :

Retornar verdadeiro

Senão:

Retornar falso

Função ClassificarMedicao(valor):

Se valor ≥ -85 :

Retornar "Normal"

Senão Se valor ≥ -100 :

Retornar "Alerta"

Senão:

Retornar "Crítico"

Função CalcularMedia(lista):

Soma todos os valores na lista e divide pelo número de valores.

Retornar média.

Função EncontrarMinimo(lista):

Encontra o menor valor na lista.

Retornar mínimo.

Função EncontrarMaximo(lista):

Encontra o maior valor na lista.

Retornar máximo.

```
Função CalcularDesvioPadrao(lista):  
    Calcula o desvio padrão dos valores na lista.  
    Retornar desvio padrão.
```

3. Parte 2: Implementação em Python Gerada por LLMs

O prompt utilizado para ambas as LLMs foi:

Crie um programa em Python para uma estação de monitoramento de potência de sinal recebido em dBm. O programa deve receber uma lista de medições, validar valores entre -120 e 0 dBm, calcular média, mínimo, máximo, desvio padrão, classificar medições em normal, alerta e crítico (Normal: ≥ -85 dBm, Alerta: entre -100 e -85 dBm, Crítico: < -100 dBm), detectar eventos críticos consecutivos e gerar um relatório. Utilize modularização com funções de responsabilidade única.

3.1. ChatGPT (GPT-5)

Python

```
from __future__ import annotations  
  
from dataclasses import dataclass  
from typing import Callable, Dict, Iterable, List, Optional, Sequence, Tuple  
import math  
import statistics  
import sys  
  
# -----  
# Modelos de dados  
# -----  
  
@dataclass(frozen=True)  
class Stats:  
    """Estatísticas básicas de um conjunto de medições."""  
    count: int  
    mean: Optional[float]  
    minimum: Optional[float]  
    maximum: Optional[float]  
    std_dev: Optional[float] # Desvio padrão populacional (pstdev)  
  
@dataclass(frozen=True)  
class Run:
```

```

        """Sequência consecutiva de eventos (p. ex., críticos)."""
        start_index: int # índice na lista validada
        end_index: int
        length: int
        values: List[float]

@dataclass(frozen=True)
class ValidationResult:
    """Resultado da validação das medições."""
    valid_values: List[float]
    invalid_entries: List[Tuple[int, float]] # (índice original, valor inválido)

# -----
# Validação
# -----

def is_valid_measurement(value: float, min_dbm: float = -120.0, max_dbm: float = 0.0) :
    """
    Verifica se uma medição está no intervalo permitido [min_dbm, max_dbm].
    """
    return min_dbm <= value <= max_dbm

def validate_measurements(measurements: Iterable[float],
                          min_dbm: float = -120.0,
                          max_dbm: float = 0.0) -> ValidationResult:
    """
    Valida medições e separa valores válidos e inválidos.
    Retorna:
        - valid_values: lista dos valores no intervalo [min_dbm, max_dbm].
        - invalid_entries: lista de tuplas (índice_original, valor) fora do intervalo.
    """
    valid: List[float] = []
    invalid: List[Tuple[int, float]] = []
    for idx, v in enumerate(measurements):
        if is_valid_measurement(v, min_dbm, max_dbm):
            valid.append(v)
        else:
            invalid.append((idx, v))
    return ValidationResult(valid_values=valid, invalid_entries=invalid)

# -----
# Estatísticas
# -----

```

```

def compute_stats(values: Sequence[float]) -> Stats:
    """
    Calcula estatísticas básicas (média, mínimo, máximo, desvio padrão populacional).
    Se a lista estiver vazia, campos numéricos serão None.
    """
    if not values:
        return Stats(count=0, mean=None, minimum=None, maximum=None, std_dev=None)

    count = len(values)
    mean = statistics.fmean(values) if count > 0 else None
    minimum = min(values) if count > 0 else None
    maximum = max(values) if count > 0 else None
    # Desvio padrão populacional (pstdev) retorna 0.0 para 1 elemento.
    std_dev = statistics.pstdev(values) if count > 0 else None
    return Stats(count=count, mean=mean, minimum=minimum, maximum=maximum, std_dev=std_dev)

# -----
# Classificação
# -----

def classify_measurement(value: float) -> str:
    """
    Classifica uma medição em:
    - 'Normal' se value >= -85 dBm
    - 'Alerta' se -100 dBm <= value < -85 dBm
    - 'Crítico' se value < -100 dBm
    """
    if value >= -85.0:
        return "Normal"
    if value >= -100.0:
        return "Alerta"
    return "Crítico"

def classify_all(values: Sequence[float]) -> List[str]:
    """
    Classifica uma lista de medições.
    """
    return [classify_measurement(v) for v in values]

def count_categories(labels: Sequence[str]) -> Dict[str, int]:
    """
    Conta quantas ocorrências há de cada categoria nas labels fornecidas.
    """
    counts: Dict[str, int] = {"Normal": 0, "Alerta": 0, "Crítico": 0}
    for lbl in labels:

```

```

        if lbl in counts:
            counts[lbl] += 1
        else:
            counts[lbl] = 1
    return counts

```

```

# -----
# Detecção de eventos consecutivos
# -----

```

```

def detect_consecutive_runs(values: Sequence[float],
                             predicate: Callable[[float], bool],
                             min_run_len: int = 2) -> List[Run]:
    """
    Detecta sequências consecutivas em 'values' onde 'predicate(value)' é True.
    Apenas sequências com comprimento >= min_run_len são retornadas.
    Os índices são relativos à lista validada (após filtragem).
    """
    runs: List[Run] = []
    n = len(values)
    i = 0

    while i < n:
        if predicate(values[i]):
            start = i
            while i < n and predicate(values[i]):
                i += 1
            end = i - 1
            length = end - start + 1
            if length >= min_run_len:
                runs.append(Run(start_index=start, end_index=end, length=length))
        else:
            i += 1

    return runs

```

```

# -----
# Relatório
# -----

```

```

def format_dbm(value: Optional[float]) -> str:
    """
    Formata valores dBm com 2 casas decimais; 'N/A' para None.
    """
    if value is None or (isinstance(value, float) and math.isnan(value)):
        return "N/A"

```

```
return f"{value:.2f} dBm"
```

```
def generate_report(original_measurements: Sequence[float],
                    validation: ValidationResult,
                    stats: Stats,
                    labels: Sequence[str],
                    runs_critical: Sequence[Run]) -> str:
    """
    Gera um relatório textual com:
    - totais, inválidos,
    - estatísticas,
    - contagem por categoria,
    - eventos críticos consecutivos (>= 2).
    Observação: índices de eventos se referem à lista de medições válidas.
    """
    total = len(original_measurements)
    invalid_count = len(validation.invalid_entries)
    valid_count = len(validation.valid_values)

    cat_counts = count_categories(labels)
    normal_c = cat_counts.get("Normal", 0)
    alert_c = cat_counts.get("Alerta", 0)
    critical_c = cat_counts.get("Crítico", 0)

    lines: List[str] = []
    lines.append("Relatório da Estação de Monitoramento de Potência (dBm)")
    lines.append("-----")
    lines.append(f"Medições totais (originais): {total}")
    lines.append(f"Medições válidas: {valid_count}")
    lines.append(f"Medições inválidas: {invalid_count}")
    if invalid_count > 0:
        lines.append("Valores inválidos (índice_original => valor):")
        for idx, val in validation.invalid_entries:
            lines.append(f"  [{idx}] => {val}")

    lines.append("")
    lines.append("Estatísticas (somente sobre medições válidas):")
    lines.append(f"  Mínimo: {format_dbm(stats.minimum)}")
    lines.append(f"  Máximo: {format_dbm(stats.maximum)}")
    lines.append(f"  Média: {format_dbm(stats.mean)}")
    lines.append(f"  Desvio padrão (populacional): {f'{stats.std_dev:.2f}' if s

    lines.append("")
    lines.append("Classificação das medições válidas:")
    lines.append(f"  Normal: {normal_c}")
    lines.append(f"  Alerta: {alert_c}")
    lines.append(f"  Crítico: {critical_c}")
```

```

lines.append("")
lines.append("Eventos críticos consecutivos (comprimento >= 2):")
if not runs_critical:
    lines.append(" Nenhum evento crítico consecutivo detectado.")
else:
    for idx, run in enumerate(runs_critical, start=1):
        seq_str = ", ".join(f"{v:.2f}" for v in run.values)
        lines.append(f" {idx}. Índices [validos]: {run.start_index}..{run.end_index}")

return "\n".join(lines)

```

```

# -----
# API principal (função utilitária)
# -----

```

```

def analyze_measurements(measurements: Sequence[float],
                        min_dbm: float = -120.0,
                        max_dbm: float = 0.0,
                        min_critical_run: int = 2) -> str:
    """
    Função de alto nível que:
    1) Valida medições.
    2) Calcula estatísticas das válidas.
    3) Classifica medições válidas.
    4) Detecta eventos críticos consecutivos.
    5) Retorna um relatório textual consolidado.
    """
    validation = validate_measurements(measurements, min_dbm=min_dbm, max_dbm=max_dbm)
    stats = compute_stats(validation.valid_values)
    labels = classify_all(validation.valid_values)
    runs_critical = detect_consecutive_runs(
        validation.valid_values,
        predicate=lambda v: classify_measurement(v) == "Crítico",
        min_run_len=min_critical_run
    )
    report = generate_report(
        original_measurements=measurements,
        validation=validation,
        stats=stats,
        labels=labels,
        runs_critical=runs_critical
    )
    return report

```

```

# -----

```



```

# Utilidades de CLI simples (opcional)
# -----

def _parse_cli_measurements(argv: List[str]) -> List[float]:
    """
    Converte argumentos da linha de comando em floats.
    Aceita valores separados por espaço ou vírgula.
    Ex.: python prog.py -85 -90 -101
         python prog.py "-80,-86,-102,0"
    """
    tokens: List[str] = []
    for arg in argv:
        # Permite "a,b,c" ou "a" isolado
        tokens.extend([t for t in arg.split(",") if t.strip() != ""])
    values: List[float] = []
    for t in tokens:
        try:
            values.append(float(t))
        except ValueError:
            print(f"Aviso: argumento ignorado (não numérico): '{t}'", file=sys.
    return values

def main() -> None:
    """
    Execução por linha de comando.
    - Se valores forem fornecidos como argumentos, usa-os.
    - Caso contrário, executa um exemplo.
    """
    argv = sys.argv[1:]
    if argv:
        measurements = _parse_cli_measurements(argv)
        if not measurements:
            print("Nenhuma medição válida fornecida por linha de comando.")
            return
    else:
        # Exemplo de uso
        measurements = [
            -78.0, -86.5, -99.9, -101.2, -103.0, -50.0, -120.0, -121.0, 1.0, -8.
            -100.0, -100.1, -85.0, -85.1, 0.0
        ]

    report = analyze_measurements(measurements)
    print(report)

if __name__ == "__main__":
    main()

```

3.2. Gemini (Gemini 3 Flash Preview)

Python

```
import statistics

def validar_medicoes(medicoes):
    """Filtra apenas valores dentro do intervalo permitido (-120 a 0 dBm)."""
    return [m for m in medicoes if -120 <= m <= 0]

def classificar_sinal(valor):
    """Classifica a potência do sinal com base nos limiares definidos."""
    if valor >= -85:
        return "Normal"
    elif valor >= -100:
        return "Alerta"
    else:
        return "Crítico"

def calcular_estatisticas(medicoes):
    """Calcula métricas estatísticas básicas."""
    if not medicoes:
        return None

    return {
        "media": statistics.mean(medicoes),
        "min": min(medicoes),
        "max": max(medicoes),
        "desvio_padrao": statistics.stdev(medicoes) if len(medicoes) > 1 else 0
    }

def detectar_sequencias_criticas(medicoes):
    """Detecta sequências de eventos críticos consecutivos."""
    sequencias = []
    atual = []

    for m in medicoes:
        if classificar_sinal(m) == "Crítico":
            atual.append(m)
        else:
            if len(atual) > 1:
                sequencias.append(list(atual))
            atual = []

    if len(atual) > 1:
        sequencias.append(atual)
```

```

    return sequencias

def gerar_relatorio(medicoes_originais):
    """Processa os dados e exibe o relatório final."""
    validas = validar_medicoes(medicoes_originais)

    if not validas:
        print("Erro: Nenhuma medição válida encontrada no intervalo de -120 a 0")
        return

    stats = calcular_estatisticas(validas)
    criticos_consecutivos = detectar_sequencias_criticas(validas)

    print("-" * 40)
    print("RELATÓRIO DE MONITORAMENTO DE SINAL")
    print("-" * 40)
    print(f"Total de medições válidas: {len(validas)}")
    print(f"Média: {stats['media']:.2f} dBm")
    print(f"Mínimo: {stats['min']} dBm")
    print(f"Máximo: {stats['max']} dBm")
    print(f"Desvio Padrão: {stats['desvio_padrao']:.2f}")
    print("-" * 40)

    print("Classificação individual:")
    for m in validas:
        print(f"    {m} dBm -> {classificar_sinal(m)}")

    if criticos_consecutivos:
        print("-" * 40)
        print(f"ALERTA: {len(criticos_consecutivos)} evento(s) crítico(s) conse")
        for seq in criticos_consecutivos:
            print(f"    Sequência: {seq}")

    print("-" * 40)

# Exemplo de uso
if __name__ == "__main__":
    # Dados de exemplo (inclui valores fora do range para teste de validação)
    dados_brutos = [-70, -90, -110, -105, -80, -125, -95, -115, -112, 10, -60]

    gerar_relatorio(dados_brutos)

```

3.3. Dados de Teste

O conjunto de dados de teste utilizado para ambos os códigos foi:

Python

```
dados_teste = [-90, -87, -95, -101, -103, -110, -75, -80, -130, "abc", -98]
```

3.4. Resultados dos Testes

ChatGPT (GPT-5)

Plain Text

```
Relatório da Estação de Monitoramento de Potência (dBm)
```

```
-----  
Medições totais (originais): 10
```

```
Medições válidas: 9
```

```
Medições inválidas: 1
```

```
Valores inválidos (índice_original => valor):
```

```
  [8] => -130.0
```

```
Estatísticas (somente sobre medições válidas):
```

```
  Mínimo: -110.00 dBm
```

```
  Máximo: -75.00 dBm
```

```
  Média:  -93.22 dBm
```

```
  Desvio padrão (populacional): 10.64
```

```
Classificação das medições válidas:
```

```
  Normal:  2
```

```
  Alerta:  4
```

```
  Crítico: 3
```

```
Eventos críticos consecutivos (comprimento >= 2):
```

```
  1. Índices [validos]: 3..5 (len=3) => [-101.00, -103.00, -110.00] dBm
```

Gemini (Gemini 3 Flash Preview)

Plain Text

```
-----  
RELATÓRIO DE MONITORAMENTO DE SINAL
```

```
-----  
Total de medições válidas: 9
```

```
Média: -93.22 dBm
```

```
Mínimo: -110.0 dBm
```

```
Máximo: -75.0 dBm
```

```
Desvio Padrão: 11.29
```

```
-----  
Classificação individual:
```

```
-90.0 dBm -> Alerta
-87.0 dBm -> Alerta
-95.0 dBm -> Alerta
-101.0 dBm -> Crítico
-103.0 dBm -> Crítico
-110.0 dBm -> Crítico
-75.0 dBm -> Normal
-80.0 dBm -> Normal
-98.0 dBm -> Alerta
```

ALERTA: 1 evento(s) crítico(s) consecutivo(s) detectado(s)!
Sequência: [-101.0, -103.0, -110.0]

4. Parte 3: Análise Comparativa da Modularização

4.1. Tabela Comparativa

Critério	ChatGPT (GPT-5)	Gemini (Gemini 3 Flash Preview)
Funções com responsabilidade única	2	1
Nomes descritivos	2	2
Evita repetição de código (DRY)	2	1
Parâmetros e retornos claros	2	2
Reaproveitamento de funções	2	1
Função principal organizando o fluxo (<code>main</code>)	2	2
Legibilidade geral e comentários úteis	2	1
Total (máx. 14)	14	9

4.2. Análise Crítica

O **ChatGPT (GPT-5)** produziu o código mais modular e bem estruturado, obtendo a pontuação máxima na tabela comparativa. Ele demonstrou uma excelente capacidade de decompor o problema em funções com responsabilidades únicas, como

`is_valid_measurement`, `validate_measurements`, `compute_stats`, `classify_measurement`, `detect_consecutive_runs` e `generate_report`. Além disso, utilizou classes `dataclass` para organizar os resultados de validação e estatísticas, o que contribui para a clareza dos parâmetros e retornos. A função `analyze_measurements` atua como uma API principal, orquestrando as chamadas para as funções menores, e há um bom reaproveitamento de funções, como `classify_measurement` sendo usada dentro de `detect_consecutive_runs`. A legibilidade é alta, com nomes descritivos e comentários úteis.

O **Gemini (Gemini 3 Flash Preview)**, embora funcional, apresentou um nível de modularização inferior. As funções `validar_medicoes`, `classificar_sinal` e `calcular_estatisticas` são bem definidas, mas a função `gerar_relatorio` acumula muitas responsabilidades, incluindo a chamada de outras funções, a formatação e a impressão do relatório. A detecção de sequências críticas também está um pouco mais acoplada à lógica de classificação dentro da mesma função, e a forma como os resultados são apresentados (impressão direta em vez de retorno de uma string formatada) dificulta o reaproveitamento. O código funciona, mas a organização poderia ser melhorada para evitar a repetição de lógica e aumentar a clareza. Por exemplo, a lógica de contagem de categorias não foi explicitamente separada em uma função, e a detecção de sequências críticas poderia ser mais genérica.

Em comparação com o pseudo-código autoral, o ChatGPT divergiu de forma positiva, estruturando a solução de maneira mais robusta e completa, introduzindo `dataclasses` e uma função de orquestração (`analyze_measurements`) que não estavam explicitamente no pseudo-código, mas que aprimoram a modularidade. O Gemini seguiu uma abordagem mais direta, mas com menos granularidade na separação de responsabilidades em algumas funções.

5. Reflexão de Engenharia: Importância da Modularização

A modularização é um pilar fundamental na engenharia de software, especialmente em sistemas reais de monitoramento como o exemplificado. Ela oferece diversas vantagens críticas:

- **Manutenção:** Módulos menores e com responsabilidade única são mais fáceis de entender, depurar e modificar. Uma alteração em uma funcionalidade específica afeta apenas um módulo, reduzindo o risco de introduzir bugs em outras partes do sistema.
- **Teste:** A capacidade de testar cada módulo isoladamente (testes unitários) é drasticamente melhorada. Isso garante que cada componente funcione corretamente antes de ser integrado ao sistema maior, facilitando a identificação e correção de falhas.
- **Reuso:** Funções bem definidas e com responsabilidades claras podem ser facilmente reutilizadas em diferentes partes do mesmo projeto ou em projetos futuros. Por

exemplo, uma função de validação de medição pode ser usada em diversos contextos onde dados de sinal são processados.

- **Trabalho em Equipe:** Em equipes de desenvolvimento, a modularização permite que diferentes membros trabalhem em módulos distintos simultaneamente, com menor chance de conflitos e maior produtividade. A interface clara entre os módulos facilita a integração do trabalho.

Em um sistema de monitoramento de telecomunicações, onde a precisão e a confiabilidade são cruciais, a modularização garante que novas funcionalidades (como novos tipos de classificação, diferentes estatísticas ou formatos de relatório) possam ser adicionadas ou modificadas com segurança e eficiência, sem comprometer a estabilidade do sistema existente.

6. Conclusão

Este estudo demonstrou que, embora ambas as LLMs sejam capazes de gerar código funcional a partir de um prompt detalhado, o ChatGPT (GPT-5) se destacou na produção de um código mais modularizado e aderente aos princípios de engenharia de software. A modularização é essencial para a construção de sistemas robustos, manuteníveis e escaláveis, e a capacidade de uma IA em gerar tal código é um indicador importante de sua utilidade em tarefas de desenvolvimento de software complexas.

7. Referências

- [1] Pseudo-código autoral: /home/ubuntu/pseudo_codigo.md
- [2] Código Python gerado pelo ChatGPT: /home/ubuntu/codigo_chatgpt.py
- [3] Código Python gerado pelo Gemini: /home/ubuntu/codigo_gemini.py
- [4] Prompt utilizado para as IAs: /home/ubuntu/prompt_ia.txt